**Report 1**

***Profir Alexandru ICA 256/2***

# Genomaly: Ethical Analysis of an Interpretable Failure-detection Assistant for Cloud Systems

**Description**

For my dissertation thesis I've chosen to study how to leverage **Genetic Programming** in **Ensembles** to aggregate fuzzy learned decisions with the goal of identifying failure points in cloud systems. **Genomaly** is the application side of this idea, where users upload / stream in batches telemetry and KPI's from a running system: metrics (CPU, memory, latency, error rates), trace summaries, and logs. The application answers the following question per analyze time-frame:

1. **Detection**: is the system in an anomalous or failure state?
2. **Diagnosis**: what kind of fault is it (resource, network, config, or code-level)?
3. **Localization**: which service, node, or metric is the most likely root cause?
4. **Explanation**: which rules and signals produced the answer?

The model consists of an ensemble of small and readable classifiers evolved using Genetic Programming, a flavor of Evolutionary Search. The scores of these deciders are fused with a fuzzy learning measure, with XAI methods (LIME, SHAP, interaction indices) to indicate which services and signals drove each decision.

The application is designed mainly for DevOps engineers, Site Reliability Engineers (SREs) and developers who run cloud or Kubernetes applications. Additionally, less experiences / technical maintainers that have no dedicate operations staff can use this application to receive plain-language explanations and support for common telemetry formats. The end users of the monitored services are indirect beneficiaries, because less downtime means more reliable public and private digital services.

Genomaly is a proposed augment for cloud operations automation, its users gaining the following perks:

1. **Faster triage**: a ranked list of suspect services and metrics instead of manual dashboard search through hundreds of series.

2. **Explainability**: explanations they can check against their own knowledge, which builds or withholds trust for good reasons.
3. **False positive mitgation**: fewer false alarms on benign load events (metrics scraping, log rotation), which reduces alert fatigue.
4. **Guidance**: for less experienced teams, a guided path into incident analysis that would otherwise need operations expertise.

**Ethical AI Principles Analysis**

The analysis is based on the following datasets that are planned for developing of this ML-based application:

- RCAEval RE1–RE3: 735 fault-injection cases on three demo microservice apps (Online Boutique, Sock Shop, Train Ticket). It has metrics, logs and traces, with fault-type, root-cause service and indicator labels. CC-BY-4.0 / MIT licenses.
- SMD: 28 real production machines, 38 anonymised metrics, 327 anomaly segments with interpretation labels. MIT licence.
- Nezha: 101 multi-modal fault cases on Online Boutique and Train Ticket, including code-level faults. MIT licence.
- AIClusterKPI: 31 cluster KPIs with 145 injected faults and 453 labelled benign load events. CC-BY-4.0.

**Principle 1 *Fairness and non-discrimination***

| **Issue** | **Possible mitigation** |
|---|---|
| **Narrow system coverage** – training data comes from three demo microservice apps on Kubernetes and one company's servers. Monoliths, serverless, VM-based or others are under-represented, so their users get worse results without knowing it. | Results should be reported per system so that users can compare with what they manage. The training process can run in **one-vs-rest** manner and also with **cross-dataset** to ensure greater system coverage. Also, we can use service-agnostic features and warn users when input falls outside the training distribution so that they are aware that the results might be misleading. |
| **Injected faults differ from real failures** – available data provides a clean test bed that does not account for real incidents format where, usually, incidents overlap and are gradual; the model might look better in evaluation than in practice. | Again cross-dataset aggregation where real incidents are collected from richer data sources. This ensures that we have real production data combined with an ML-ready source that accounts for other cases. |

| | |
|---|---|
| **Scarce and unbalanced data** – given that failures are present but not that frequent there is a rarity among labelled failure data. Cost-sensitive learning cannot add diversity, and a model can score high accuracy by predicting normal behavior. | Generate synthetic minority samples with imbalanced-learning oversampling ([Borderline-]SMOTE, ADASYN) on the real windowed features. Balancing will apply only on the training folds in order to prevent data leakage. Combined with cost-sensitive GP fitness measures it should allow more accurate representation of failure cases. |
| **Common-fault bias –** oversampling can amplify common faults (such as resource faults) making them dominate the distribution. Also, samples interpolated from a few rare cases repeat those cases' behavior, resulting in no additional knowledge regarding failure behavior. | Oversampling should happen per fault type and system, not globally. Results should be compared with/without oversampling and synthethic sample should be compared against real individuals. |
| **Unequal telemetry richness** – well-equipped teams usually have a telemetry stack that provides metrics, logs, and traces for their applications. Amateur or small teams often have metrics only, built-in by the solutions they port or minimal telmetry. | A metric-only model will be available for all users, while multi-modality (logs and traces processing) will be available on need. Model performance will be reported per modality set so that the model capabilities are clear. |

**Principle 2 *Safety and security***

| **Issue** | **Possible mitigation** |
|---|---|
| **Dual use for attack reconnaissance** – an attacker with stolen or scraped telemetry from a target can use the service to finds its weakest services and fault types, then aim a targeted attack (e.g., resource exhaustion, DDoS). | Users/organizations will be verified to claim authenticity while also complying to extensive terms of use to prevent misusage. Proof of ownership of the analyzed system will be required, such as signed telemetry export or tokens installed in cluster. Requests will be rate-limited and analysis logs will be kept with respect to GDPR. |
| **Data breach and personal data in telemetry** – log traces can contain user IDs, IP addresses and other PII. A breach would expose both customer infrastructure details and personal data of their end users. | Zero retention by default: telemetry is analyzed in memory and deleted once the resulte is returned, so client data stays the client's responsibility. Data is kept only when a tenant opts in to contribute it for retraining. |
| **Model extraction and IP exposure –** the GP members are readable formulas. If they leak, attackers can learn the exact | Resulted GP members, thresholds, and fuzzy measure stay private. Users get explanations at the level of signals and services, not the internal |

| | |
|---|---|
| thresholds that trigger an alert and can stay just below them, and also competitors can copy the model. | formulas. Full internals are open only to internal reviewers and independent auditors under agreement. |
| **Data and feedback poisoning** – malicious tenants could upload crafted telemetry or false corrections so that retraining learns to ignore a certain attack pattern. | Mitigating this would imply that retraining is never done on user data or feedback. Tenant data is kept separate from the global training set, every retrained model being revalidate against a fixed, trusted benchmark. |
| **Harmful errors –** a false negative lets an outage go unnoticed; frequent false positives cause alert fatigue, so operators start ignoring real alerts. | Operating points can be tuned along with users and results would be reported on both feedback/no-feedback rates. Confidence with each alert would be a good indicator for treating errors. |
| **Silent degradation over time by concept drift –** new releases, traffic patterns and technologies change the telemetry, and accuracy can drop without anyone noticing. | Monitor feature distributions per tenant and warn on drift. Retrain on a schedule, offline, with version models and guards against poisoning risk. |

**Principle 3 *Transparency and explainability***

| **Issue** | **Possible mitigation** |
|---|---|
| **Unfaithful explanations** – post-hoc explainers (e.g., SHAP on gradient boosting) explain a model, not the system, and can point on the wrong cause. In incident response, a wrong cause wastes time during outage. | Explanations are built directly into the model: the fuzzy aggregator splits exactly into member contributions (Shapley values) and pairwise interactions. |
| **Explanations are too technical, or that expose the model –** an internal member or an interaction index means little to a beginner in the field, also posing a risk to leak the IP. | Explanations would revolve around the decision, not the model. Member contributions will be translated into signal and service contributions. The default decision view is a ranked list plus a sentence such as "*latency of car rose sharply while redis memory grew*". The expert view adds contribution scores per metric and the strongest signal pairs that reinforce each other. Formulas stay internal. |
| **Transparency about training data and scope –** users cannot judge trust without knowing what the model was trained on: three demo apps, test bed faults, one company's servers. | A model card and datasheet would serve as transparent data about the model's data: datasets, licenses, systems, fault types, known gaps, and per-system results. |
| **Hidden uncertainty** – a single plain | The decision view will always contain candidate |

| | |
|---|---|
| explanation (*root-cause: X*) reads as certain even when the evidence is weak or split across services. | scores sorted decreasingly, agreement between members, and a “low confidence” flag. Show “no confident diagnosis” instead of forcing an answer. |
| **Explaining an accuracy-interpretability trade-off** – the interpretable ensemble may score a few points below black-box models. Users should know what they gain and what they give up. | Report benchmark results against strong baselines openly, including where the solution loses. Explain why a checkable rule can be worth a small loss accuracy. |

**Principle 4 *Human oversight and determination***

| **Issue** | **Possible mitigation** |
|---|---|
| **Over-reaching automation** – an assistant that also restarts pods, scales services, or rolls back releases could worsen an incident (e.g., restart the wrong service or cascade during a misdiagnosis). | The application would have no write access to the user’s infrastructure. The service would ONLY suggest next steps; a human decides and acts. Any future remediation feature would require explicit, per-action human approval. |
| **Automation bias** – under on-call pressure, operators may accept the top-ranked cause without checking it. | Show the evidence (metrics, rules) next to every claim, and link to the raw telemetry. Hidden uncertainty would be made visible and a *verify* step could be added to make sure that the decision is verified by an actual human. |
| **Contestability without self-modification**- users must be able to say *this diagnosis is wrong*, but letting the model change itself from each correct would be unsafe. | Record each correction with its incident. Use corrections immediately to suppress repeat alerts for that tenant only. Feed reviewed corrections into the next scheduled retraining. |
| **Accountability for missed incidents** – when an outage is missed or misdiagnosed, it is unclear who is responsible: the tool, the operator, or the vendor. | State in the terms of use that this application is decision support and that the operator remains responsible for action. Keep audit logs of the model version, a hash of the input and output of each analysis, without raw telemetry. Post-incident reviews can then trace what the system showed. |

**Conclusions**

The proposed application, **Genomaly**, is a viable candidate for anomaly detection and fault-failure analysis on cloud deployed services. The above analysis found out the following strong and weak points.

**Strong points**

- Transparency is the most achievable principle, because explanation is built into the method. The learned fuzzy measure gives Shapley and interaction attributions that can be scored against ground-truth labels instead of judged by eye.
- Human oversight is easy to guarantee by design: the system has no write access and only advises.

**Weak points**

- Fairness across systems is hard to guarantee. Public labeled data covers only a few demo application and one production fleet, so performance on other stacks is unknown and must be stated as a limitation.
- Safety is the most delicate principle. The tool is dual-use, telemetry can carry personal data, and readable models are easier to extract and evade.
- Scarce labeled data is the main limit. Oversampling balances the classes but cannot invent failure behavior that is not in data.
- Transparency and IP protection must be balanced The application explains decisions (signals and services) while keeping the model's internals private. This reduces the tension but does not remove it, since attributions still reveal what the model watches.

**Instructor questions**

The following questions have been asked after the instructor has reviewed the first version of this report. Question and answers are listed below.

**Which of the safety and security concerns present the highest risk of impact, why, and how do you plan to mitigate them? In detail.**

In my opinion, from all listed safety & security risks, two stand-out as greater risks:

1. **Dual use for attack reconnaissance**: given the sensitive domain that my approach would act in, malicious users / attackers could use stolen telemetry, logs, traces to target vulnerabilities exposed by our service. A mitigation proposal, as stated in the possible mitigation proposal, would be some sort of authentication and authorization protocol which would enforce who can access the service and which data passed to the service would be considered as *signed & genuine*. Public-private key authentication, identity provider for user management, and signed telemetry data are all viable candidates for mitigating dual-use.
2. **Data breach and personal data in telemetry**: this overlaps with the **Privacy** ethical principle, but its attack footprint resides in security vulnerabilities. Our app would use

zero-retention by default, with client data passed for analysis being scanned in memory and deleted once the analysis is finished. Clients could opt-in for for data retention that can be used to retrain the model under the hood of the application such that the application can improve over time considering usage patterns of cloud applications from real examples. If clients decide to provide data for retraining, the terms of use would state that the data passed is the responsibility of both the client and the service: the client would make sure that no sensitive data is passed during analysis and the service would make sure that the stored client data is encrypted securely in an user access-gated data store so that it is less prone to data breaches.